

exp0414_simplexNet 工程说明文稿

现状梳理与工程化建议

Shuheng Zhang

2026 年 4 月 20 日

摘要

本文稿面向 `memory_lm` 仓库中的 `exp0414_simplexNet` 子项目，给出一份中文为主的工程说明。本文不把该目录描述为完整的强化学习系统，而是将其定位为一个围绕 Simplex Memory Network (SMN) 的结构实现与实验验证原型。文稿分为三部分：其一，准确说明当前代码已经实现的能力、训练路径和测试覆盖；其二，指出当前工程封装上的缺口，尤其是 `checkpoint` 生命周期、本地日志、配置契约与交互确认机制；其三，给出后续工程化重构建议，包括核心模块拆分、配置组织以及基于 `PySide` 的训练/测试界面规划。全文只将当前仓库中可以直接核实的内容写作“现状”，并将未来方案集中放入“建议与规划”部分，避免把未实现功能误写成既成事实。

目录

1	项目定位	2
2	当前代码结构	2
2.1	主要脚本与职责	2
2.2	SMNModule、SMNFitter 与 MLPFitter 的关系	3
3	当前训练与测试流程	4
3.1	训练主链路	4
3.2	当前任务类型	4
3.3	测试覆盖	4
4	当前已具备的能力	5
4.1	SMN 结构实现已完整落地	5
4.2	支持多种输入输出形态	5
4.3	提供 MLP baseline 对照	5
4.4	支持 <code>state_dict</code> 级别的模型序列化接口	5
4.5	存在 RL 使用示例，但仅处于示例层	6

1 项目定位	2
5 当前缺口与封装问题	6
5.1 缺少本地 checkpoint 生命周期管理	6
5.2 缺少配置兼容性比较与替换确认	6
5.3 缺少持续追加的本地运行日志	6
5.4 配置层与执行层已有漂移	7
6 工程化建议	7
6.1 建议的核心模块拆分	7
6.2 checkpoint 与确认机制建议	7
6.3 配置组织建议	8
7 配置与 GUI 规划	8
7.1 训练与测试界面应独立，但风格统一	8
7.2 GUI 的职责边界	8
8 结论	8

1 项目定位

`exp0414_simplexNet` 的核心目标，是实现并验证一种新的前馈网络结构：Simplex Memory Network (SMN)。从当前代码路径看，该目录的主实验入口是 `run.py`，其默认工作方式是：

```
params.yaml → build_dataset → SMNFitter.fit → MLPFitter.fit (optional) → plot
```

这一定义了它当前的真实身份：结构原型 + 监督学习实验封装。它不是一个现成的完整强化学习项目，也不是一个带有持久化训练状态管理的工程系统。

仓库中确实存在 `examples/smn_for_rl.py`，其中展示了如何把 `SMNModule` 作为 DQN 的 Q-network 或 REINFORCE 的 policy network 使用。但该文件仅是示例接口，并不是 `exp0414_simplexNet` 的主实验入口，也不代表当前主流程已经转向 RL 训练。

因此，对该目录最准确的表述应为：

- 它首先是一个新的网络结构实现项目；
- 它目前通过合成监督学习任务验证表达能力与训练表现；
- 它保留了未来接入 RL 的可能性，但 RL 仍然处在示例层，而非主线工程层。

2 当前代码结构

2.1 主要脚本与职责

当前目录的主要文件可以按职责划分为以下几类。

路径/模块	当前职责
<code>src/config.py</code>	配置数据类 <code>Config</code> 与 YAML 加载逻辑。
<code>src/data.py</code>	合成任务定义与数据集构造，包括 1D/2D 回归任务。
<code>src/graph.py</code>	单纯形格点、势函数诱导有向图、拓扑层级等几何骨架。
<code>src/smn_fitter.py</code>	同时包含 <code>SMNModule</code> 与 <code>SMNFitter</code> ；即模型定义和高层训练封装混在同一文件。
<code>src/mlp_fitter.py</code>	MLP baseline 的训练封装，与 <code>SMNFitter</code> 接口对齐。
<code>src/plot.py</code>	可视化输出，包括散点图、loss 曲线与四图对比。
<code>run.py</code>	读取配置、构造数据、训练 SMN/MLP、保存图像。
<code>tests/test_smn.py</code>	针对 <code>SMNModule</code> 的单元测试。
<code>examples/smn_for_rl.py</code>	将 <code>SMNModule</code> 作为 RL 函数逼近器使用的示例。

2.2 SMNModule、SMNFitter 与 MLPFitter 的关系

当前实现中存在两层不同抽象。

第一层：网络结构本身 `SMNModule` 是一个标准的 PyTorch `nn.Module`。它负责：

- 构造 `SimplexMemoryGraph`；
- 注册输入归一化所需的 `_x_min/_x_max` buffers；
- 维护边权、核心节点偏置与输出偏置；
- 完成按 DAG 拓扑层级推进的前向传播。

第二层：实验封装 `SMNFitter` 是面向实验的高层包装器。它在 `SMNModule` 之上补了如下能力：

- 训练循环调用；
- 训练/验证损失缓存；
- 默认 demo 数据；
- 预测；
- 图像输出。

`MLPFitter` 则提供与 `SMNFitter` 尽量相同的外层接口，以便在 `run.py` 中充当 baseline。

这一设计让实验推进很快，但也带来一个明显后果：模型定义、训练过程、结果缓存与展示逻辑耦合在一起。这在原型阶段可以接受，但对后续 checkpoint 恢复、GUI 调用和 RL 接入都不够理想。

3 当前训练与测试流程

3.1 训练主链路

`run.py` 的实际流程可以概括为以下步骤：

1. 读取 `params.yaml`，构造 `Config`；
2. 根据 `task_name`、`n_in`、`x_bounds` 等设置调用 `build_dataset`；
3. 构造 `SMNFitter` 并执行 `fit()`；
4. 若 `compare_mlp=true`，则额外训练 `MLPFitter`；
5. 调用 `plot()` 输出单模型或 SMN/MLP 对比图；
6. 将图像保存在 `runs/<date>/<exp_name_timestamp>/`。

3.2 当前任务类型

`src/data.py` 当前提供的是合成监督学习任务，而不是环境交互型任务。已注册任务包括：

- 1 输入 1 输出： `sin`, `sin_mix`, `poly_wave`, `piecewise`
- 1 输入 2 输出： `sin_cos`
- 2 输入 1 输出： `sin_sum`, `sin_product`, `quadratic`
- 2 输入 2 输出： `trig_2d`

这进一步说明，当前 0414 的主实验仍然是函数逼近问题，而不是 MDP 训练问题。

3.3 测试覆盖

`tests/test_smn.py` 当前主要覆盖 `SMNModule` 的基础行为，包括：

- SISO/MIMO 形状正确性；
- 1D 输入压缩形状；
- 输出范围受最终 `tanh` 限制；
- `x_bounds` 归一化行为；
- `x_bounds` 长度不匹配时抛错；
- 参数数量为正；
- 架构字符串格式；
- 反向传播梯度可流动；

- 若干 (n, m) 组合的 smoke test;
- `state_dict` save/load roundtrip。

这意味着当前测试重点是结构级正确性，而不是完整训练系统的状态管理、结果重现或多轮实验恢复。

4 当前已具备的能力

基于当前代码，可以确认该项目已经具备以下能力。

4.1 SMN 结构实现已完整落地

`graph.py` 已经实现：

- 单纯形格点枚举；
- 输入面 `F_in`、输出面 `F_out`、共享中间面 `F_mid`；
- 势函数驱动的有向最近邻边；
- 前驱/后继表；
- 拓扑层级计算与验证。

`SMNModule` 则将这一几何骨架变为实际可训练网络。

4.2 支持多种输入输出形态

当前实现不仅支持 SISO，也支持更一般的 MIMO 场景。输入归一化通过每通道 `x_bounds` 完成；输出维度由 `n_out` 控制。这一点已经在配置、数据生成和测试中得到体现。

4.3 提供 MLP baseline 对照

`MLPFitter` 被设计为与 `SMNFitter` 拥有近似一致的接口，因此可以在同一训练脚本中直接进行结构比较。当前 `run.py` 默认就是以 SMN 为主角、MLP 为对照。

4.4 支持 `state_dict` 级别的模型序列化接口

作为标准 `nn.Module`，`SMNModule` 已支持 `state_dict()` 与 `load_state_dict()`。测试中也有 roundtrip 检验。这意味着模型参数本身是可保存、可恢复的。

不过，需要强调的是：当前支持的是接口层面的序列化能力，而不是完整的本地 **checkpoint** 生命周期管理系统。这一点将在后文说明。

4.5 存在 RL 使用示例，但仅处于示例层

`examples/smn_for_rl.py` 中包含 DQN 与 REINFORCE 风格的样例，说明 `SMNModule` 可以作为状态到 Q 值、或状态到策略 logits 的映射器使用。这一层证明了该结构与 RL 的接口是可行的。

但当前目录并未提供一个以 RL 为主线的正式训练入口，因此不应把该示例误判为该目录已经实现了完整 RL 框架。

5 当前缺口与封装问题

5.1 缺少本地 checkpoint 生命周期管理

当前训练逻辑在 `smn_fitter.py` 与 `mlp_fitter.py` 中都维护了一个内存中的 `best_state`，训练完成后会把最优参数重新加载回模块。但这一过程只发生在内存里，不会自动写入磁盘 **checkpoint**。

因此，当前系统不存在以下工程能力：

- 自动发现已有本地权重；
- 兼容时优先加载本地权重；
- 不兼容时展示差异并由用户确认；
- 初始化新模型后自动建立可追踪的 **checkpoint** 历史；
- 中断后继续训练。

5.2 缺少配置兼容性比较与替换确认

从当前代码看，训练系统不会检查“本地已有参数是否与当前配置相容”，也不会在发生初始化、替换或覆盖前弹出确认。对研究型实验来说，这意味着已有结果很容易被忽略；对未来 RL 训练来说，这则是更高风险的问题。

后续如果要把该目录升级为可恢复训练系统，至少应当支持：

- **checkpoint** 元信息记录；
- 配置差异比较；
- `state_dict` key/shape 差异报告；
- 初始化或替换前的确认弹窗；
- 决策结果写入日志。

5.3 缺少持续追加的本地运行日志

当前目录会在 `runs/` 下输出图片，但缺少一份持续追加的本地日志文件，用于记录每次运行的配置、加载来源、差异检查结果、训练过程与保存动作。这使得实验历史难以追溯，也不利于后续排查“哪次训练用了哪个模型、在哪一步被替换”。

5.4 配置层与执行层已有漂移

`Config` 与 `params.yaml` 中存在一些当前未真正接入主流程的字段, 例如:

- `model_type`
- `window_width`
- `window_hold`

`run.py` 当前仍然固定训练 SMN, 并在需要时附带训练 MLP baseline, 而不是根据 `model_type` 选择主模型。同样, 窗口训练参数已经出现在配置中, 但未实际驱动训练主链路。对工程系统而言, 这种“配置表面大于实际实现”的状态容易误导使用者。

6 工程化建议

6.1 建议的核心模块拆分

若要把 `exp0414_simplexNet` 从“实验原型”推进为“可恢复、可审计、可扩展”的工程系统, 更清晰的主干应当拆成四个核心模块:

problem

定义问题本身。当前阶段可承载监督学习任务定义; 若未来接入 RL, 则承载 MDP、状态动作空间和采样接口。

model

定义网络结构本身。SMN 的几何图与 `SMNModule` 都应归于这一层。

train

定义训练流程, 包括 `optimizer`、`loss`、`checkpoint`、日志和恢复逻辑。

test/eval

定义单元测试、评估流程、结果分析与回归验证。

相应地, `plot`、`logs`、`checkpoints`、`docs` 则应作为辅助层, 而不应反向侵入训练主流程。

6.2 checkpoint 与确认机制建议

后续 `checkpoint` 机制应遵循如下保守原则:

1. 启动时先检查本地已有权重;
2. 若本地权重与当前配置兼容, 则优先加载, 而不是重新初始化;
3. 若不兼容, 则明确展示差异, 说明不兼容发生在哪些字段、哪些参数键或哪些张量形状上;
4. 只有在用户确认后, 才允许初始化新模型或替换旧模型;
5. 所有初始化、替换、加载与保存动作都写入本地追加日志。

这套机制的目标不是“自动化越多越好”, 而是“对已有实验成果尽量保守, 对覆盖行为尽量显式确认”。

6.3 配置组织建议

后续配置不宜继续只依赖一份混合式 `params.yaml`。更稳妥的方案是拆成三类配置：

- `model.yaml`：只描述模型结构与输入输出边界；
- `train.yaml`：只描述训练超参数、checkpoint 策略、日志策略；
- `eval.yaml`：只描述测试/评估所需的模型来源、任务、指标与输出选项。

其中，模型结构字段应在训练与测试之间共享定义，以避免配置契约漂移。

7 配置与 GUI 规划

7.1 训练与测试界面应独立，但风格统一

如果后续要给该项目补上桌面交互层，推荐采用 PySide 构建两个独立窗口：

- 训练窗口：用于选择/比对 checkpoint、编辑训练配置、启动训练、查看日志和过程曲线；
- 测试/评估窗口：用于加载模型、选择任务或评估数据、运行评估并查看结果。

这两个界面应当共享统一的视觉系统，例如字体、色彩、卡片布局、确认弹窗样式和日志面板风格，但不应合并为一个职责过于混杂的“大一统窗口”。

7.2 GUI 的职责边界

需要强调，GUI 只能是交互壳层，而不应承载核心训练逻辑。较合理的职责分工是：

- GUI 负责读取/写入 YAML、展示差异、弹出确认框、启动任务、展示日志与结果；
- 后端模块负责真实的训练、恢复、保存、评估与错误处理。

只有这样，后续 CLI 批处理、自动实验脚本或其他前端入口才不会被 GUI 实现反向绑定。

8 结论

`exp0414_simplexNet` 当前的价值，不在于它已经成为一个完整的训练平台，而在于它已经把一种新型网络结构 SMN 实实在在地编码成了可运行、可训练、可测试、可对照的原型系统。它完成了最关键的一步：证明“结构”本身是可以落地的。

但若要继续向长期使用的实验系统推进，当前还缺少若干工程层能力，尤其是 checkpoint 生命周期、差异比较、确认机制、追加日志、配置契约与 GUI 分层。这些问题并不否定当前目录的研究价值，恰恰说明它正处在一个清晰的下一阶段入口上：

从结构原型与实验封装 → 可恢复、可审计、可交互的工程系统

因此，本文稿的基本判断是：0414 已经是一个很好的网络结构研究原型，但它的下一步不应只是“多跑几组实验”，而应是把训练、测试、配置、持久化和交互层系统性地补齐。